Solving Flatland with ASP

Tom Schmidt, Ariane Lehmann

University Potsdam

Abstract. In this paper we will introduce the train scheduling problem and its representation in flatland. Additionally we will present our approaches to solve such problems using Answer Set Programming and compare their results.

Keywords: Artificial Intelligence · Train Scheduling.

1 Train scheduling problem

1.1 Flatland

Today research regarding train scheduling for real-world applications is still relevant. One framework, which allows experimentation with different approaches is Flatland [1, 2]. Flatland represent railway networks as 2D grids environments. The size of these environments can be freely set. Trains are represented by Agents, which are defined by their position and their direction. The destination of every agent is a grid cell symbolizing a train station. Each grid cell can only be occupied by one agent at a given time. To keep an agents movement on the train tracks, restrictions in the transitions between neighbouring grid cells are used.

Agent actions supported by Flatland are moving forward, selecting a turn, halting on the current cell and no operation. A forward move on a dead end is interpreted as a turn to the opposite direction. The availability of these actions is given by the corresponding transition map of the cell the agent is currently on and the direction the agent is facing.

To help agents make their decisions each agents observes the scenario. Flatland supports multiple observation spaces, the easiest being the global observation. When using the global observation space every agent is provided with a global view of the full environment. Other observation spaces are the local observation and the tree observation. Additionally to its observations every agent also receives multiple reward signals. The rewards encourage valid moves and a timely arrival of each agent individually, but also collectively.

The complexity of the scenario can be further increased by other Flatland features such as railway signals, varying train speeds, malfunctions and different observation spaces.

A visualization of the railway environment and the actions of every agent can be created with the provided render framework.

1.2 Problem definition

In this paper we will focus on the most basic version of the train scheduling problem. Which means finding a path for each agent from its start cell to its target, while avoiding collisions. The problem can be defined by the following properties:

- All agents have to reach their target.
- Collisions have to be avoided.
- All agents have exactly one start and one destination cell.
- All agents spawn at the same time.
- All agents move with a constant speed of 1 cell per time step.
- There are no malfunctions.
- The entire track is known during planning.
- All paths are entirely pre-calculated.

2 Encodings

In our approaches to solve train scheduling with Answer Set Programming (ASP) using the tools from Potassco [3] we had two main goals. The encoding should be as robust as possible, finding a solution for all instances and corner cases. And, given enough time the solution should always be optimal. Because of these goals we had to made concessions in other area, e.g. performance increases due to simplification. These downsides will be further discussed in Section 3. Furthermore, while flatland uses a combination of directions and positions for its description of transitions and actions, we tried to use a solely position-based approach. The solution of our approach would then be converted back to flatland interpretable actions at the end. All our tools, instances and encodings can be found in our repository [4]

2.1 Instances

All our instances are generated in Python using the Flatland framework and exported as a `.pkl` file. These instance files can then be converted into `.lp` instances using our converter. All `.lp` instances contain 7 different types of atoms.

- `agent(A)`, represents all agents in the instance with `A` being the agent id
  - `starting(A,(Y,X))`, the starting position of agent `A`
  - `direction(A,D)`, the starting direction of agent `A`
  - `target(A,(Y,X))`, the target position of agent `A`
- `cell(Y,X)`, representing all cells in the instance
  - `transraw((Y,X),D1,D2))`, valid transition when the cell was entered from direction `D1` and exited in direction `D2`
  - `trans_count((Y,X),D,N)`, `N` number of valid transitions of the cell when entered from direction `D`

Directions are numbered as follows: north: 0, east: 1, south: 2, west: 3. Because Flatland always uses (Y,X) instead of (X,Y) to describe their coordinates, we will also stick with the former.

2.2 Helper encoding

To allow for easier modification and iterative improvements during development, we split a static part of our encoding into an extra `helper.lp` encoding (Listing 1.1). This encoding provides some general utilities, which can be used in different encodings and approaches.

The `conv(DIR,(DY,DX))` facts in lines 3-9 provide an easy way to convert a direction `DIR`, as found in the instances for example, to relative coordinates `(DY,DX)`, e.g. 0 (north) → (-1,0) (origin of coordinates is at the top-left). The described above instances provide facts in the form of `transraw((Y,X),IN,OUT)`, which states, that cell `(Y,X)` can be exited in direction `OUT` when previously entered from direction `IN`. `trans(C1,C2,C3)`, derived in lines 12-14, contains the same information just applied to a triple of cells instead of using directions. This allows for a mostly position-based approach. To later address all possible turns, the relative position changes are defined in line 17 using `turn(DY,DX)`.

Listing 1.1. helper.lp

```
1  % directions
2  % north
3  conv(0,(-1,0)).
4  % east
5  conv(1,(0,1)).
6  % south
7  conv(2,(1,0)).
8  % west
9  conv(3,(0,-1)).
10
11 % transitions
12 trans((Y+DY1,X+DX1),(Y,X),(Y+DY2,X+DX2))
13       :- transraw((Y,X),D1,D2),
14       conv(D1,(DY1, DX1)), conv(D2,(DY2,DX2)).
15
16 % turns:
17 turn(-1,1). turn(1,1). turn(1,-1). turn(-1,-1).
```

2.3 Scheduler encoding

Our scheduler encoding implements a position-based approach to solving flatland instances and can be split into 3 different parts: generation, constraints and conversion. The generation part can be seen in Listing 1.2. Its purpose is to generate possible paths for all agents.

Listing 1.2. generation part of the scheduler

```
1  % step limit
2  #const slimit=20.
3
4  step(1..slimit).
5
6  % start positions
7  at(A,(Y,X),0) :- agent(A), starting(A,(Y,X)).
8
9  % generate positions
10 0{at(A,(Y,X),S) : cell(Y,X)}1 :- agent(A), step(S).
```

The maximum number of steps is defined via the slimit constant (default = 20) in line 2. Using slimit step(S) are generated to represent the different time steps.

Line 7 and 10 show the generation of positions and indirectly paths for each agent. To keep track of the positions, the predicate `at(A,(Y,X),S)` is used. Stating agent `A` is at cell `(Y,X)` at step `S`. First, the positions at step 0 are copied from the starting positions given by the instance. Secondly for each agent and step at least one cell is assigned.

The second part of our encoding now removes all paths, which do not result in a valid solution, using the constraints in Listing 1.3 and 1.4. To make sure the agents achieve their objective the first two constraints in lines 13 and 15 in Listing 1.3 are introduced.

While the first constraint guarantees, that agents finish at their target, the second one prohibits agents from occupying any other position once they reached their target. The latter is based on the property of flatland, that agents can despawn once their target is reached. Should this not be the case the last constraint should be removed to allow the encoding to solve specific corner cases.

In a flatland scenario, two types of collisions can occur. Vertex collisions (two agents at the same position at the same step) and edge collisions (two agents collide while swapping positions). The constraints in lines 18-23 prohibit both collision types.

Listing 1.3. constraint part of the scheduler

```
12 % finish at target
13 :- not at(A,(Y,X),_), target(A,(Y,X)), agent(A).
14 % dont move away from target
15 :- at(A,C,S1), at(A,_,S2), S1<S2, target(A,C), agent(A).
16 % no collisions
17 % vertex
18 :- at(A1,(Y,X),S), at(A2,(Y,X),S),
19     step(S), agent(A1), agent(A2), A1!=A2.
20 % edge
21 :- at(A1,(Y1,X1),S), at(A2,(Y2,X2),S),
22     at(A1,(Y2,X2),S-1), at(A2,(Y1,X1),S-1),
23     step(S), agent(A1), agent(A2), A1!=A2.
```

The movement of all agents is constrained by the train track layout of the instance, i.e. its transitions. Since all valid transitions are denoted as cell triples `trans(C1,C2,C3)` as stated above in the description of the helper encoding, this approach causes problems when agents are waiting. To fix this, `entered_from(A,C,S)` atoms are introduced. These keep track of the previous (different) cell `C` for each agent `A` at each time step `S`. That means waiting of an agent does not change cell C. The implementation of `entered_from/3` can be found in our repository [4].

All remaining criteria for a valid path are shown in Listing 1.4. First, all disconnected paths are removed, i.e. positions for each agent at each time step represented by `at(A,C,S)` have to be either adjacent to or the same as the position of the previous step. Secondly, the path should be continuous, so all paths with missing positions are removed. The last constraint in lines 41 and 42 guarantees, that all paths respect the given transitions.

Listing 1.4. constraint part of the scheduler, continuation

```
34 % only valid actions
35 % move to neighbors or wait
36 :- at(A,(Y1,X1),S-1), at(A,(Y2,X2),S),
37     |Y1-Y2|+|X1-X2|>1, step(S), agent(A).
38 % continuous
39 :- not at(A,_,S-1), at(A,_,S), step(S), agent(A).
40 % transitions
41 :- entered_from(A,C1,S-1), at(A,C2,S-1), at(A,C3,S),
42     C2!=C3, not trans(C1,C2,C3), step(S), agent(A).
```

The last part of our encoding, the conversion, implements the conversion from our generated position-based path back to Flatland's action-based domain. This conversion is split into 4 parts corresponding to the Flatland actions. 'left' and 'forward' can be seen in Listing 1.5 and Listing 1.6 respectively and 'right' and 'stop' in Listing 1.7.

To transform the valid solution back into flatland interpretable actions, `action(A,AC,S)` is introduced (agent `A` performs action `AC` at step `S`). Since the "none" action is unused in our solution it can be ignored. The left turn with action number 1 is shown in lines 47-54. A turn equals a change in position by the values represented in `turn(DY,DX)` in two steps (one on the x-axis and one on the y-axis). Does the sum of `DY` and `DX` equal 0 and the first step was done along the x-axis, then a left turn occurred. Similarly a left turn occurred for a sum other than 0 and a first step along the y-axis.

To avoid 'false' left turns (track turns left, but no switch), `trans_count(C,D,N)`, given by the instance, keeps track of the number of transitions `N` per cell `C` for each direction `D` the agent could enter from. If this value is equal to or bigger than 2, the cell contains a switch. Using this, all 'true' left turns and therefore necessary 'left' actions can be found by solely checking the positions of each agent at all steps.

For the 'forward' action (2) two cases have to be addressed, moving forward on normal tracks (lines 57-62) and on switches (lines 65-70). In the case of no switches ($N = 1$), any change in position is automatically a forward movement and action. If there are switches ($N \geq 2$), movements only along the X-axis (Y coordinate remains constant) or Y-axis (X coordinate remains constant) show a forward movement and result in a 'forward' action.

Listing 1.5. 'left' action conversion

```
44 % actions:
45 % none: 0
46 % left: 1
47 action(A,1,S) :- entered_from(A,(Y,X-DX),S),
48       at(A,(Y,X),S), at(A,(Y+DY,X),S+1), turn(DY,DX),
49       DY+DX=0, trans_count((Y,X),D,N), conv(D,(0,-DX)),
50       N>=2.
51 action(A,1,S) :- entered_from(A,(Y-DY,X),S),
52       at(A,(Y,X),S), at(A,(Y,X+DX),S+1), turn(DY,DX),
53       DY+DX!=0, trans_count((Y,X),D,N), conv(D,(-DY,0)),
54       N>=2.
```

Listing 1.6. 'forward' action conversion

```
56 % forward: 2
57 action(A,2,S) :- entered_from(A,(Y+DY,X+DX),S),
58     at(A,(Y,X),S), at(A,(Y1,X1),S+1), Y!=Y1,
59     trans_count((Y,X),D,N), conv(D,(DY,DX)), N=1.
60 action(A,2,S) :- entered_from(A,(Y+DY,X+DX),S),
61     at(A,(Y,X),S), at(A,(Y1,X1),S+1), X!=X1,
62     trans_count((Y,X),D,N), conv(D,(DY,DX)), N=1.
63
64 % forward on switches
65 action(A,2,S) :- entered_from(A,(Y,X+DX),S),
66     at(A,(Y,X),S), at(A,(Y,_),S+1),
67     trans_count((Y,X),D,N), conv(D,(0,DX)), N>=2.
68 action(A,2,S) :- entered_from(A,(Y+DY,X),S),
69     at(A,(Y,X),S), at(A,(_,X),S+1),
70     trans_count((Y,X),D,N), conv(D,(DY,0)), N>=2.
```

The last two actions can be found in Listing 1.7. The right turn (3) is the same as the left turn, just the order of steps corresponding to the sum of `DY` and `DX` are switched. The 'wait' action (4) in line 81 can simply be inferred by no change in the position of subsequent steps.

Listing 1.7. 'right' and 'stop' action conversion

```
71 % right: 3
72 action(A,3,S) :- entered_from(A,(Y,X-DX),S),
73     at(A,(Y,X),S), at(A,(Y+DY,X),S+1), turn(DY,DX),
74     DY+DX!=0, trans_count((Y,X),D,N), conv(D,(0,-DX)),
75     N>=2.
76 action(A,3,S) :- entered_from(A,(Y-DY,X),S),
77     at(A,(Y,X),S), at(A,(Y,X+DX),S+1), turn(DY,DX),
78     DY+DX=0, trans_count((Y,X),D,N), conv(D,(-DY,0)),
79     N>=2.
80 % stop: 4
81 action(A,4,S) :- at(A,C,S), at(A,C,S+1).
```

The minimization in Listing 1.8 tries to find an optimal solution with no useless "wait" actions and the fastest path to the target for all agents. The last two lines generate our output, which can be interpreted by our other tools.

Listing 1.8. minimization and output

```
83 #minimize{1,S,A: at(A,_,S)}.
84 #show at/3.
85 #show action/3.
```

2.4 Improvements

Subsequently we tried to improve our encoding by reducing the number of possible choices generated in line 10 in Listing 1.2. We developed two different versions.

In the first version (`1_1.lp`) the generation of possible positions (`at/3`) is limited to neighbouring cells with tracks instead of all cells with tracks of the instance, seen in Listing 1.9.

Listing 1.9. changes in version 1_1

```
0{at(A,(Y2,X2),S) : cell(Y2,X2), at(A,(Y1,X1),S-1),
    |Y1-Y2|+|X1-X2|<=1}1 :- agent(A), step(S).
```

This approach was expanded on in the second version (`1_2.lp`) by limiting possible positions to cells with valid transitions. To be able to allow agents to remain in their position the transitions had to be expanded. The changes of this version can be found in Listing 1.10.

Listing 1.10. changes in version 1_2

```
% add transitions for waiting
trans(C1,C2,C2) :- trans(C1,C2,_).
% limit to valid transitions
0{at(A,C3,S) : trans(C1,C2,C3), entered_from(A,C1,S-1),
    at(A,C2,S-1)}1 :- agent(A), step(S).
```

At first glance both versions cause some constraints to become unnecessary. For example the constraint that guarantees moves only to neighbouring cells is included in the generation of version 1_1. But as seen in Section 4.5 the removal of these additional constraint can have unexpected consequences.

A completely different approach was to splits waiting and moving. First possible `stay(A,S)` predicates are generated, which denote that agent `A` stays in its previous position at step `S`, which results in the positions in the following line. For all remaining steps, possible positions (moves) can then be derived by the valid transitions.

Since this second approach was slower than the versions above and scaled worse than the original encoding, we decided to stop further development in that direction. More information regarding this approach can be found in our repository [4].

3 Difficulties

During the development of our approach, we faced multiple challenges. These obstacles include various aspects of the problem-solving process, ranging from managing the conversion between different representations to effectively handling the complexities inherent in track layouts. Additionally, as we tried to extend our solution to larger instances, concerns regarding scalability emerged.

3.1 Creating custom flatland instances

We encountered multiple challenges while creating custom flatland instances in order to test our encoding and corner cases. The first challenge was to create instances with multiple trains and more train stations as trains. This occurred since the line generator supplied by flatland only uses the minimum amount of train stations needed. Meaning there are 2 stations for an instance with a single train so that it has a start and a target station. But for instances with 2 or more trains, train stations can be the start of one train and the goal for another. This results in the number of stations always being equal to the number of trains, even if there were more stations defined as goals. So in order to create other instances like the one in Figure 1 a new custom line generator needed to be created.

The next challenge we encountered while creating custom instances was that flatland was not allowing stations in dead-ends with trains starting in the direction that leaves the dead end. But unintuitively, it is allowed to create trains that start in a dead end and face into the dead end. In Figure 1 all trains had to start facing outwards as a result.

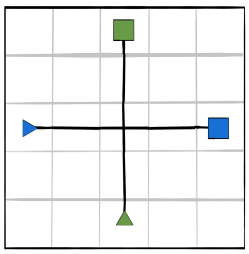


Fig. 1. Instance 5x5x2-wait

3.2 Choosing the Horizon

The next challenge arose while testing our custom instances as some of these instances were clearly solvable but could not be solved in flatland. The reason behind this was the standard horizon of the flatland solver. The default horizon of the flatland solver is calculated based on the number of steps of the longest path to reach the goal of all trains disregarding collisions. As a result, instances with trains waiting a high number of time-steps or having to take long detours because of collision avoidance require a higher horizon. In order to automate the testing a different way of calculating the horizon was needed. After testing we decided to have a minimal number for small instances that increases slowly with instance size.

3.3 Converting the solution to flatland

Our last challenge was to convert our solution back to flatland in order to visualize it. As flatland uses actions instead of paths we needed to convert our path into actions. The initial problem was that movements on a turning track are considered a 'forward' actions and actions 'left' or 'right' are only used on switches. In order to differentiate between going left or right because of a curve and choosing left or right on a switch, we decided to track the number of valid transitions as explained in Section 2.3 and implemented in Listings 1.5, 1.6 and 1.7.

4 Results

We benchmarked our different versions of our approach on multiple the custom instances. We tested our 3 main version and the 2 variations with reduced constraints. While the main versions are represented by their version number, the variations are denoted by the suffix 'nc'. All encodings, instances and results can be found in our repository [4].

In order to compare all encodings we measured the time to solve and ground, the number of conflicts and the number of choices. This was done for each encoding while searching for the first solution and the optimal solution. All our encodings found a solution for all instances, where a solution was possible, except for instance 20x20x1-snake. On this instance the grounding ran out of memory with the provided 32GB. The implications of this result are discussed in Section 5. The following comparisons are mainly focused on the case of finding the first solution, since the performance of all encodings scale similarly whether they are used for finding the first or optimal solution. Missing bars in the diagram are caused by times under $0.003s$, values to small to be measured consistently.

4.1 Solving time

Figure 2 and Figure 3 show the solving time of all encodings for each instances. Figure 2 shows the solving time in seconds while Figure 3 shows the required time in a logarithmic scale. The results show, that as expected, the solving time increases with the complexity of the instance. All encodings took the longest time to find a solution for the 7x7x4-circle instance, where all 4 agents move in a big circle, made-up of 4 smaller circles, in order to reach their goals.

Surprising are the solving times for the Multicircle instances for version 1_0, as all 4 instances have the same grid size and number of rails but an increasing number of agents. But against expectations the solving time does not increase with the increasing number of agents. The other versions on the other hand behave as expected and require more time, the more agents there are.

Over all we can see that encoding 1_0 is by far the slowest on most instances, while encoding 1_1 and encoding 1_2 are mostly even or with only small differences. The most complex instance 7x7x4-circle allows us to assume the scalability of our encodings. With 1_1 requiring half the time of 1_0 and 1_2 requiring half the time of 1_1. This behavior has to be confirmed with further testing.

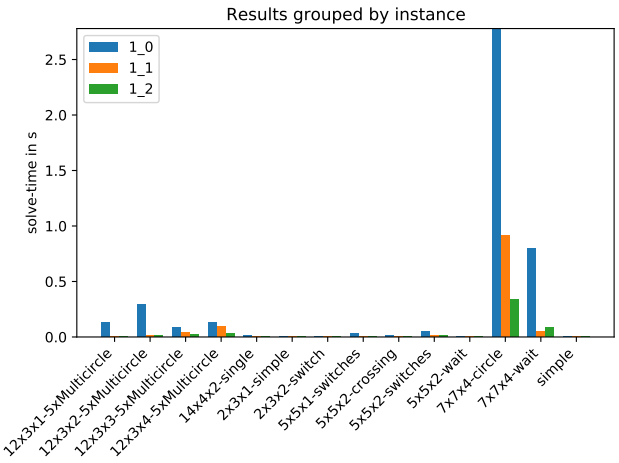


Fig. 2. solving time in seconds

Figure 4 shows the solving time of each encoding for an optimal solution in logarithmic scale. As expected, in comparison to 3 all encodings take longer to find an optimal solution but their relations remain the same. Similarly version 1_0 stays the slowest overall, while one of versions 1_1 and 1_2 is the fastest depending on the instance.

4.2 Grounding time

Figure 5 shows the grounding time for each instance. We can clearly see on the Multicirle instances, that the grounding time scales linearly with the number of agents. As expected instances with the same grid and number of agents take the same time to ground, as seen on the instances 7x7x4-circle and 7x7x4-wait. Overall version 1_0 has to perform the most and version 1_2 the least grounding. Important to note is, that compared to the solving time, the grounding time makes up by far the biggest part of the overall run-time.

4.3 Number of choices

Figure 6 and Figure 7 show the number of choices during solving. We can see that this number strongly depends on the size of the grid, number of agents and the complexity of the tracks. These results strongly correlate with the results regarding the solving time. This shows that the number of choices are the main indicator for the required solving time. Overall our efforts to reduce the number of choices with each version were effective, especially version 1_1 compared to version 1_0.

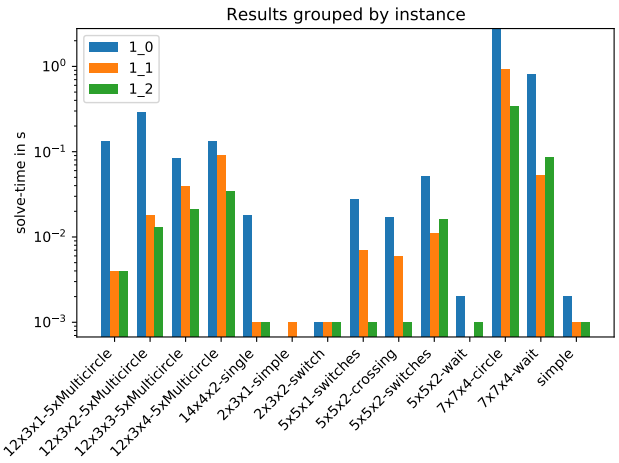


Fig. 3. solving time in seconds (log-scale)

4.4 Number of conflicts

Figure 8 shows the number of conflicts during solving. We see that the number of conflicts is the highest for the complex 7x7x4-circle instances. Surprising are the results for version 1_0 on the Multicircle instances, where the number of conflicts drop with the increasing number of agents. Overall version 1_0 is again worst even if it beats version 1_1 on the Multicircle instance with 4 agents. The best version depends on the instance, with no clear winner between version 1_1 and 1_2.

4.5 Versions with reduced constraints

If we look at the results of the versions with reduced constraint in Table 1 we can see that version 1_1_nc is not much different from version 1_1. Version 1_2_nc on the other hand trades grounding time for solving time and achieves generally better performance than the original 1_2. More results and plots can be found in our repository [4]

instance	1_1	1_1_nc	1_2	1_2_nc
12x3x1-5xMulticircle	0.004 (0.777)	0.022 (0.793)	0.004 (0.696)	0.121 (0.135)
12x3x4-5xMulticircle	0.092 (3.992)	0.022 (4.021)	0.034 (3.536)	1.087 (0.859)
7x7x4-circle	0.916 (4.417)	0.834 (4.435)	0.337 (3.394)	2.427 (0.706)
7x7x4-wait	0.053 (4.467)	0.255 (4.457)	0.087 (3.394)	0.400 (0.717)

Table 1. solving (grounding) time in seconds for selected instances

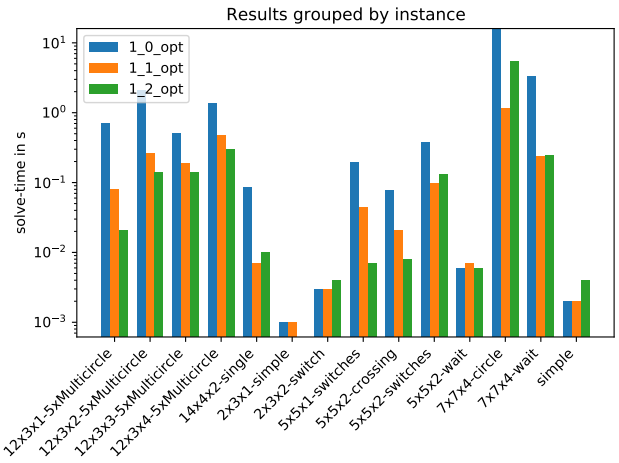


Fig. 4. solving time in seconds for an optimal solution (log-scale)

5 Conclusion

Overall our approach satisfies all our previously set goals. All edge-cases are solvable and optimal solutions can be found. As expected due to missing simplifications our approach struggles on large instances with big horizons due to many possible choices. This could be solved by better estimations of the required horizon or using our encodings with strategies like incremental solving. Another idea would be to use another 'simplified' approach for long 'inter-city' distances and use our encoding for more complicated but smaller scenarios like marshalling yards or train stations. Additionally most of the run-time is caused by grounding. In the real world railway tracks do not change regularly, which would allow the use and reuse of pre-grounded instances. When instances can be pre-grounded version 1_1 or 1_2, depending on the instance, will perform the best. Otherwise version 1_2_nc should be used.

References

1. Mohanty, S., Nygren, E., et al.: Flatland-RL : Multi-Agent Reinforcement Learning on Trains. ArXiv, abs/2012.05893. (2020)
2. Flatland documentation, https://flatland.aicrowd.com/intro.html, last accessed 18. March 2024
3. Potassco homepage, https://potassco.org/, last accessed 18. March 2024
4. Project repository, https://github.com/tzschmidt/Railway-Scheduling/tree/main, last accessed 18. March 2024

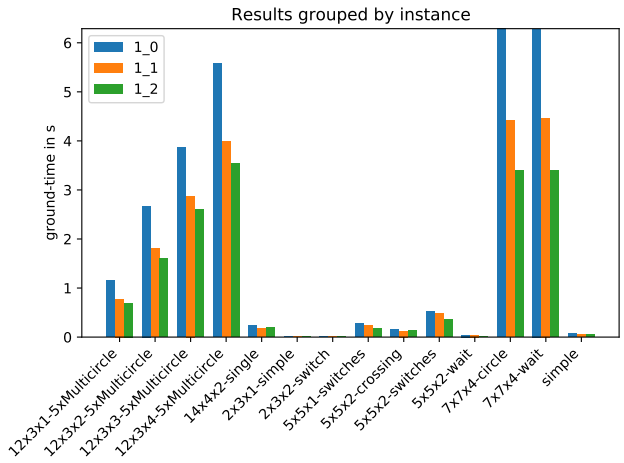


Fig. 5. grounding time in seconds

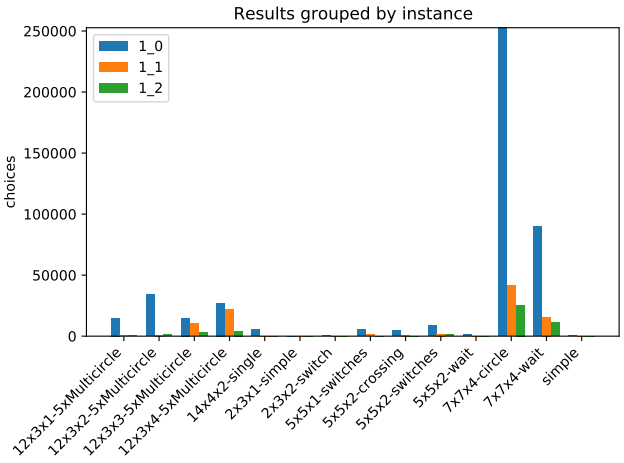


Fig. 6. number of choices

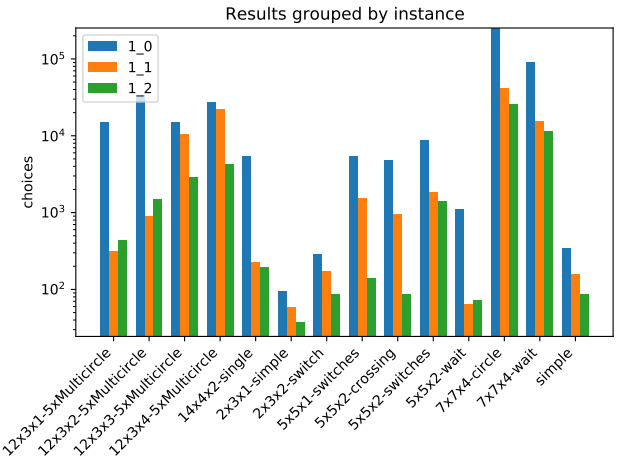


Fig. 7. number of choices (log-scale)

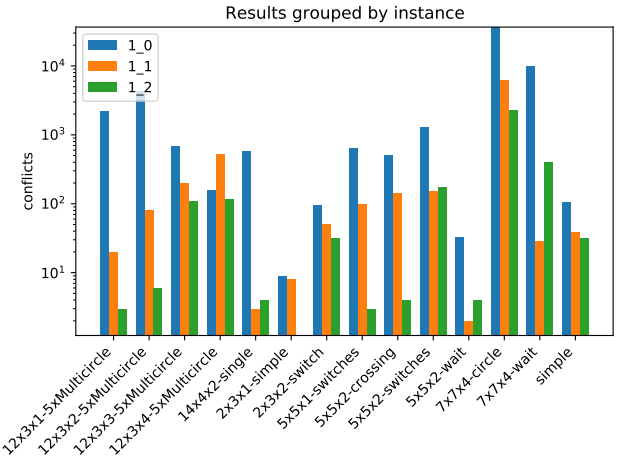


Fig. 8. number of conflicts (log-scale)